

Minimum Cost/Path Strategy

Table of Contents

Working Example	1
Algorithm	1
Primary variables.....	1
Key Idea.....	1
Concrete Example 1.....	2
Concrete Example 2.....	4

Working Example

Inputs:

- **Source:** Vertex A
- **Graph:** Defined below

```
V = 1:6;  
E = [ 1 2 1;  
1 3 7;  
1 4 6;  
2 3 2;  
3 4 8;  
3 5 3;  
4 5 4;  
4 6 6;  
5 6 4 ];
```

Directed Graph Visualization (Minimum Cost Algorithm)

Algorithm

Primary variables

- temp_cost (1 x n) - cost at each vertex so far (could change)
- perm_cost (1 x n) - minimum possible cost at each vertex (can't change)
- perm_vertices (1 x n) - a logical vector that is true at index i if vertex i is permanent.
- paths (1 x n cell array) the i-th object in paths is the min cost path from the source to vertex i.

Key Idea

Step 1: Initialize all primary variables

- The temp_cost and perm_cost start with all infinity values.
- No vertex is permanent initially.
- All paths start as an empty vector.

- Assign the source vertex a temp_cost of 0.

Step 2: Select the non-permanent vertex with the smallest temp_cost. Let's call it v.

Step 3: Make v permanent and copy its temp_cost to perm_cost.

Step 4: Look for all of v's non-permanent neighbors and

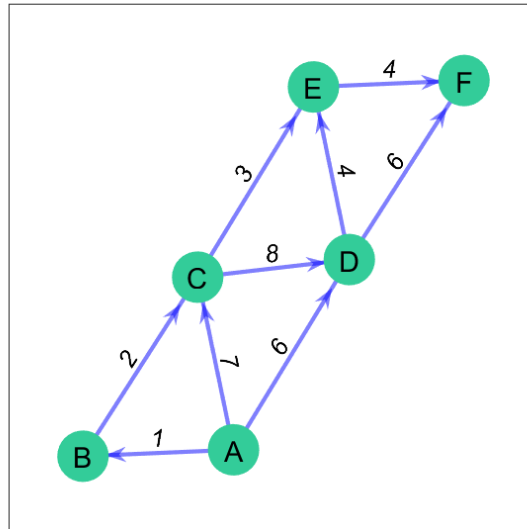
- Compute the cost to move from v to each of these neighbors
- Update that neighbor's temp_cost.
- Save each of these neighbor's current path.

Repeat Steps 2-4 until all vertices are marked as permanent.

Concrete Example 1

Compute the minimum path to all vertices from A in the graph below:

Directed Graph Visualization (Minimum Cost Algorithm)



Graph cost matrix:

0	1	7	6	0	0
0	0	2	0	0	0
0	0	0	8	3	0
0	0	0	0	4	6
0	0	0	0	0	4
0	0	0	0	0	0

--- Algorithm: Execution ---

Source Vertex: A

Temporary Costs: [0 Inf Inf Inf Inf Inf]

Permanent Vertices: [false false false false false false]

Permanent Costs: [Inf Inf Inf Inf Inf Inf]

paths: { [1], [], [], [], [], [], }

non-permanent vertex w/ min-cost: A

non-permanent neighbors of A: B, C, D

cost to move from A -> B = 1 (currently = Inf) update!

paths: { [1], [1 2], [], [], [], [], }

cost to move from A -> C = 7 (currently = Inf) update!

paths: { [1], [1 2], [1 3], [], [], [], }

cost to move from A -> D = 6 (currently = Inf) update!

paths: { [1], [1 2], [1 3], [1 4], [], [], }

REPEAT

Temporary Costs: [0 1 7 6 Inf Inf]

Permanent Vertices: [true false false false false false]

Permanent Costs: [0 Inf Inf Inf Inf Inf]

paths: { [1], [1 2], [1 3], [1 4], [], [], }

non-permanent vertex w/ min-cost: B

non-permanent neighbors of B: C

cost to move from B -> C = 3 (currently = 7) update!

paths: { [1], [1 2], [1 2 3], [1 4], [], [], }

REPEAT

Temporary Costs: [0 1 3 6 Inf Inf]

Permanent Vertices: [true true false false false false]

Permanent Costs: [0 1 Inf Inf Inf Inf]

paths: { [1], [1 2], [1 2 3], [1 4], [], [], }

non-permanent vertex w/ min-cost: C

non-permanent neighbors of C: D, E

cost to move from C -> D = 11 (currently = 6) don't update!

paths: { [1], [1 2], [1 2 3], [1 4], [], [], }

cost to move from C -> E = 6 (currently = Inf) update!

paths: { [1], [1 2], [1 2 3], [1 4], [1 2 3 5], [], }

REPEAT

Temporary Costs: [0 1 3 6 6 Inf]

Permanent Vertices: [true true true false false false]

Permanent Costs: [0 1 3 Inf Inf Inf]

paths: { [1], [1 2], [1 2 3], [1 4], [1 2 3 5], [], }

non-permanent vertex w/ min-cost: D

non-permanent neighbors of D: E, F

cost to move from D -> E = 10 (currently = 6) don't update!

paths: { [1], [1 2], [1 2 3], [1 4], [1 2 3 5], [], }

cost to move from D -> F = 12 (currently = Inf) update!

paths: { [1], [1 2], [1 2 3], [1 4], [1 2 3 5], [1 4 6], }

REPEAT

Temporary Costs: [0 1 3 6 6 12]

Permanent Vertices: [true true true true false false]

Permanent Costs: [0 1 3 6 Inf Inf]

paths: { [1], [1 2], [1 2 3], [1 4], [1 2 3 5], [1 4 6], }

```

non-permanent vertex w/ min-cost: E
  non-permanent neighbors of E: F
  cost to move from E -> F = 10 (currently = 12) update!
  paths: { [1], [1 2], [1 2 3], [1 4], [1 2 3 5], [1 2 3 5 6], }

```

REPEAT

```

Temporary Costs: [0 1 3 6 6 10]
Permanent Vertices: [true true true true true false]
Permanent Costs: [0 1 3 6 6 Inf]
paths: { [1], [1 2], [1 2 3], [1 4], [1 2 3 5], [1 2 3 5 6], }

```

```

non-permanent vertex w/ min-cost: F
  non-permanent neighbors of F: NONE

```

REPEAT

ALL VERTICES are Permanent - DONE

OUTPUT:

perm_cost =

```

    0     1     3     6     6    10

```

paths =

1x6 cell array

```

    {[1]}    {[1 2]}    {[1 2 3]}    {[1 4]}    {[1 2 3 5]}    {[1 2 3 5 6]}

```

--- Algorithm Completed ---

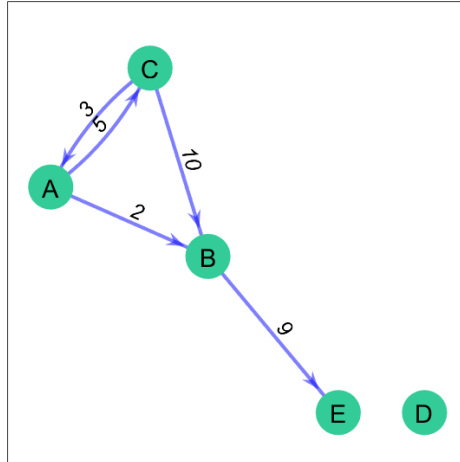
Shortest Costs from Vertex A:

VERTICES	MIN_COST_FROM_A	PATH_FROM_A
"A"	0	"A"
"B"	1	"A -> B"
"C"	3	"A -> B -> C"
"D"	6	"A -> D"
"E"	6	"A -> B -> C -> E"
"F"	10	"A -> B -> C -> E -> F"

Concrete Example 2

Compute the minimum path to all vertices from C in the graph below:

Directed Graph Visualization (Minimum Cost Algorithm)



Graph cost matrix:

0	2	5	0	0
0	0	0	0	9
3	10	0	0	0
0	0	0	0	0
0	0	0	0	0

--- Algorithm: Execution ---

Source Vertex: C

Temporary Costs: [Inf Inf 0 Inf Inf]
 Permanent Vertices: [false false false false false]
 Permanent Costs: [Inf Inf Inf Inf Inf]
 paths: { [], [], [3], [], [], }
 non-permanent vertex w/ min-cost: C
 non-permanent neighbors of C: A, B
 cost to move from C -> A = 3 (currently = Inf) update!
 paths: { [3 1], [], [3], [], [], }
 cost to move from C -> B = 10 (currently = Inf) update!
 paths: { [3 1], [3 2], [3], [], [], }

REPEAT

Temporary Costs: [3 10 0 Inf Inf]
 Permanent Vertices: [false false true false false]
 Permanent Costs: [Inf Inf 0 Inf Inf]
 paths: { [3 1], [3 2], [3], [], [], }

non-permanent vertex w/ min-cost: A
 non-permanent neighbors of A: B
 cost to move from A -> B = 5 (currently = 10) update!
 paths: { [3 1], [3 1 2], [3], [], [], }

REPEAT

Temporary Costs: [3 5 0 Inf Inf]
 Permanent Vertices: [true false true false false]
 Permanent Costs: [3 Inf 0 Inf Inf]

paths: { [3 1], [3 1 2], [3], [], [], }

non-permanent vertex w/ min-cost: B
non-permanent neighbors of B: E
cost to move from B -> E = 14 (currently = Inf) update!
paths: { [3 1], [3 1 2], [3], [], [3 1 2 5], }

REPEAT

Temporary Costs: [3 5 0 Inf 14]
Permanent Vertices: [true true true false false]
Permanent Costs: [3 5 0 Inf Inf]
paths: { [3 1], [3 1 2], [3], [], [3 1 2 5], }

non-permanent vertex w/ min-cost: E
non-permanent neighbors of E: NONE

REPEAT

Temporary Costs: [3 5 0 Inf 14]
Permanent Vertices: [true true true false true]
Permanent Costs: [3 5 0 Inf 14]
paths: { [3 1], [3 1 2], [3], [], [3 1 2 5], }

non-permanent vertex w/ min-cost: D
non-permanent neighbors of D: NONE

REPEAT

ALL VERTICES are Permanent - DONE

OUTPUT:

perm_cost =

3 5 0 Inf 14

paths =

1x5 cell array

{[3 1]} {[3 1 2]} {[3]} {0x0 double} {[3 1 2 5]}

--- Algorithm Completed ---

Shortest Costs from Vertex C:

VERTICES	MIN_COST_FROM_C	PATH_FROM_C
"A"	3	"C -> A"
"B"	5	"C -> A -> B"
"C"	0	"C"
"D"	Inf	"None"
"E"	14	"C -> A -> B -> E"